

Activities

Teaching Note: Edison Robot - Barcodes vs. EdScratch (Obstacle Avoidance)

Class Prompt / Hook:

Class, do you remember our barcodes from Week 1? Today, we are going to learn about the actual program behind those barcodes, starting with obstacle avoidance!

Overview

When we scanned the “Obstacle Avoidance” barcode in Week 1, Edison executed a pre-programmed script stored directly inside its internal firmware. In EdScratch, we write the step-by-step logic ourselves to achieve the same movement.

Comparison Summary

Feature	Barcode Scanning (“Obstacle Avoidance”)	Custom Coding in EdScratch
How It Works	Reads a fixed sequence stored in Edison’s firmware.	Downloads your step-by-step block code via the USB cable.
Sensor Setup	Automatically enables infrared (IR) emitters and receivers.	Requires turning on IR detection explicitly using setup blocks.
Control & Customization	None. Edison moves, detects, turns, and continues on a pre-set loop.	High. You decide how far to turn, how fast to drive, or whether to sound an alarm.
Logic Structure	Hidden “black box” algorithm.	Explicit loop structure using forever, if...then blocks, and obstacle detected events.

Key Discussion Points for Students

1. Barcode (Pre-made Shortcut):

The barcode tells Edison to run a hidden, built-in program. Edison does the thinking for us.

2. EdScratch (Programmer Control):

In EdScratch, **you** are the programmer. You control the exact sensing, decision-making, and physical reaction when Edison encounters an obstacle.

How do robots see

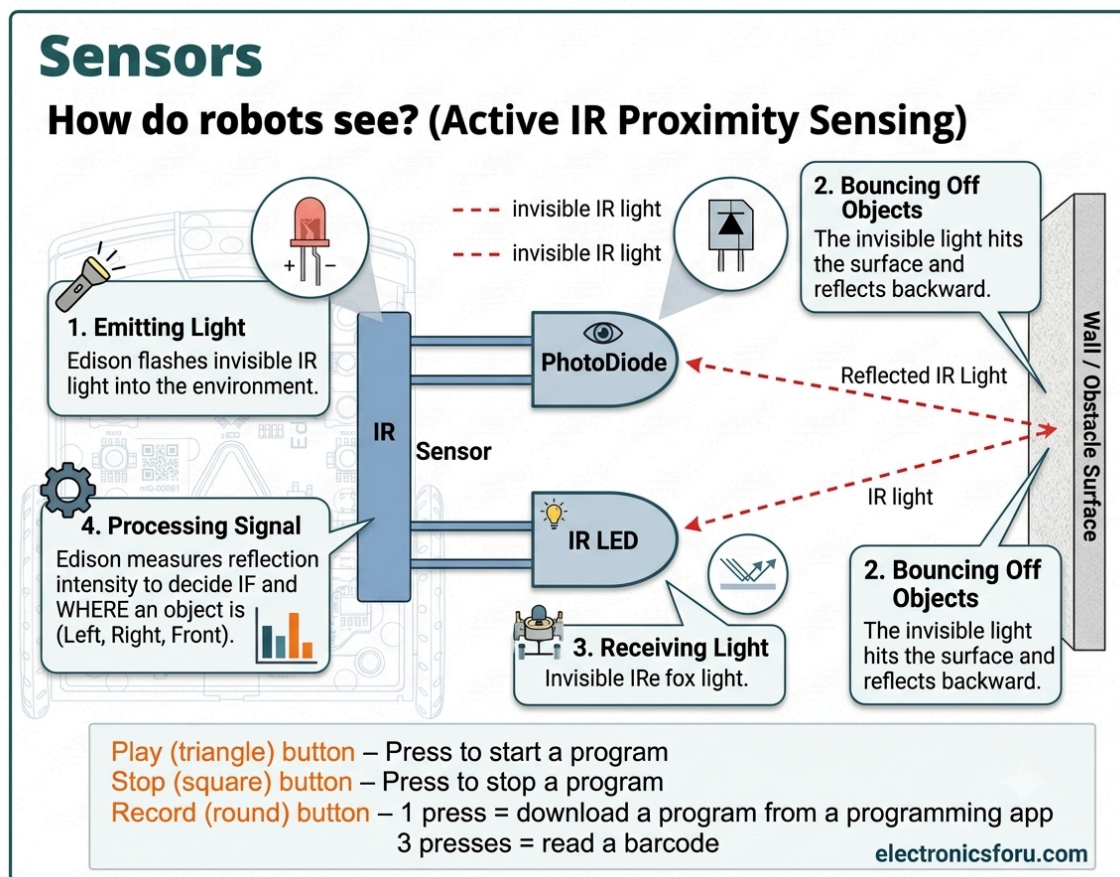


Figure 1: IR Sensor Diagram

Edison V3 Hardware Mapping: How Robots “See”

Conceptual Component	Diagram	Edison V3 Physical Component	How It Works on Edison V3
IR LED (<i>Emitter</i>)		Left & Right Infrared LEDs (<i>Front outer edges</i>)	Emits invisible infrared light forward into the environment. Edison pulses these LEDs to search for obstacles.
Object / Obstacle		Wall, hand, box, or track barrier	The physical surface directly in front of Edison that bounces the transmitted IR light back toward the sensors.
PhotoDiode (<i>Receiver</i>)		Left & Right Light Sensors (<i>Front inner corners</i>)	Detects and measures the strength of the returning reflected IR light.
IR Sensor (<i>Controller</i>)		Edison Microcontroller Board	Reads the sensor input to determine if an object is ahead and which side (left, right, or straight ahead) it is on.

Step-by-Step Sensing Process

1. **Emitting Light:** Edison flashes its **Infrared LEDs** forward.
2. **Bouncing Off Objects:** Invisible light hits an obstacle surface and reflects back.
3. **Receiving Light:** The **Light Sensors** (acting as photodiodes) catch the reflected IR signal.
4. **Processing Signal:** The **Microcontroller** determines the reflection intensity. If it exceeds the threshold, EdScratch triggers the obstacle detected block (allowing Edison to stop, turn, or play a sound).

“How does a robot ‘sees’ without actual eyes? It works just like a bat using echolocation, but with invisible light!

1. **Flash:** Edison fires out an invisible beam of light from its front IR LEDs—like a secret flashlight beam you can’t see.
2. **Bounce:** When that beam hits an obstacle, it bounces right off.
3. **Catch:** Edison’s light sensors act as the receivers, catching the light as it bounces back.
4. **Think:** Edison’s brain measures how strong that reflection is. High reflection? Obstacle close! Low reflection? Clear path ahead!

So, when you write your EdScratch code today, you aren’t just turning motors—you’re telling Edison’s brain what to do when it catches that bouncing light!”

Teaching Sequence / Activity Steps

1. **Review Week 1:** Ask students to recall how Edison reacted when scanning the “Obstacle Avoidance” barcode.
2. **Deconstruct the Behavior:** Ask students to list the exact physical steps Edison takes when it encounters an obstacle (e.g., *Drive forward → Detect obstacle → Stop → Backup/Turn → Resume driving*).
3. **Replicate in EdScratch:** Guide students to build the program block-by-block in EdScratch using the IR detection setup, forever loop, and conditional blocks.